

Tech Challenge — Fase 2

Pipeline híbrido para análise da alfabetização no Brasil

Relatório de implementação — Camadas Silver e Gold

Arquitetura medalhão · AWS S3 · Base dos Dados / BigQuery

1. Contexto e objetivo

O projeto constrói uma plataforma de dados (Lakehouse na AWS) para análise do Indicador Criança Alfabetizada (INEP / Base dos Dados), seguindo a arquitetura medalhão. A Fase 1 entregou a camada **Bronze** (ingestão dos dados brutos em Parquet no S3). Este relatório documenta a implementação das camadas **Silver** (dados tratados e integrados) e **Gold** (datasets analíticos), além dos testes, da documentação e das práticas de FinOps e monitoramento.

2. Arquitetura e fluxo de dados

O fluxo segue: Base dos Dados / BigQuery → Bronze (S3, Parquet) → Silver (profiling, qualidade, padronização e integração) → Gold (datasets analíticos) → consumo (dashboards, machine learning e apoio a políticas públicas). Cada camada é gravada em Parquet particionado por data, isolando dados brutos, tratados e analíticos.

A camada Silver executa, por tabela: **leitura** da partição mais recente da Bronze, **data profiling**, aplicação das **regras de qualidade** (separando registros válidos dos que vão para a **quarentena**), **transformação** (padronização de tipos e chaves e consolidação das metas) e **escrita** em silver/, com relatório de qualidade em quality/.

3. Módulos e arquivos criados

Camada compartilhada — src/common/

- `config.py` — configuração AWS/bucket e prefixos das camadas.
- `logger.py` — logger padronizado para observabilidade.
- `s3_io.py` — leitura da partição mais recente e escrita de Parquet particionado no S3.

Camada Silver — src/silver/

- `reader.py` — leitura das tabelas Bronze.
- `profiling.py` — linhas, schema, nulos, duplicidades e estatísticas.
- `quality.py` — regras de qualidade e separação válidos/inválidos.
- `transform.py` — limpeza, padronização e consolidação das metas.
- `upload.py` — gravação da Silver, quarentena e relatório.
- `pipeline.py` — orquestração da camada.

Camada Gold — src/gold/

- `reader.py` — leitura das tabelas Silver.
- `transform.py` — construção dos datasets analíticos.
- `upload.py` — gravação da Gold.
- `pipeline.py` — orquestração da camada.

Testes e documentação

- `tests/test_silver_gold.py` — 8 testes de lógica com dados sintéticos (sem AWS).

- docs/data_catalog.md, docs/camadas_silver_gold.md, docs/finops.md e diagrama Mermaid em docs/arquitetura.md.

4. Tabelas da camada Silver

Tabela Silver	Origem Bronze	Descrição
alunos	alunos	Microdados tratados, sem duplicidade de chave.
alfabetizacao_municipio	municipio	Indicadores municipais padronizados.
alfabetizacao_uf	uf	Indicadores estaduais padronizados.
metas	meta_brasil + meta_uf + meta_municipio	Metas consolidadas em formato longo (uma linha por ano-alvo).

Padronizações aplicadas: nomes de colunas normalizados; ano como inteiro; id_municipio como código IBGE (sem o artefato .0); sigla_uf em maiúsculas; taxas/proporções/metass como float; e remoção controlada de duplicidades por chave lógica.

5. Regras de qualidade

Regra	Condição
ano_obrigatorio	ano não nulo e numérico
taxa_entre_0_100	taxa_alfabetizacao entre 0 e 100
proficiencia_nao_negativa	proficiencia maior ou igual a 0
sigla_uf_2_caracteres	sigla_uf com exatamente 2 caracteres
id_municipio_ibge_7_digitos	id_municipio com 7 dígitos (IBGE)
meta_entre_0_100	meta_alfabetizacao entre 0 e 100
duplicidade de chave	reportada por tabela
completude	percentual de preenchimento dos campos críticos

Registros que falham em qualquer regra são desviados para a **quarentena** (quality/<tabela>_quarentena/) com a coluna _quality_errors indicando as regras violadas; os válidos seguem para silver/. Um relatório JSON com contagens e completude é gravado em quality/reports/ e localmente em reports/.

6. Datasets da camada Gold

Dataset Gold	Origem	Descrição
indicador_alfabetizacao_municipio	alfabetizacao_municipio	Indicador por município, rede e série.
comparativo_metass_resultados	metass	Taxa realizada vs. meta: gap_para_meta e atingiu_meta.
evolucao_temporal_indicador	metass	Evolução da taxa por localidade ao longo dos anos.

7. FinOps, monitoramento e aplicação em IA

FinOps — otimização de custos

- Parquet comprimido + particionamento por data (partition pruning reduz o volume escaneado).
- Arquitetura serverless (S3 + Athena sugerido), sem cluster persistente: custo proporcional ao uso.
- Leitura apenas da partição mais recente e quarentena única, evitando reprocessamento.
- Recomendação de S3 Lifecycle Policies para arquivar partições Bronze antigas.

Monitoramento

Logging padronizado registra início, fim, volume e falhas de cada etapa. As pipelines consolidam tabelas com sucesso/erro e falham explicitamente quando há erro, facilitando alertas. Relatórios de qualidade dão observabilidade de dados.

Aplicação em IA

- Predição da taxa de alfabetização por município.
- Análise de desigualdade educacional e clusters de vulnerabilidade.
- Priorização de políticas públicas por gap até a meta.

8. Como executar

```
python -m src.bronze.pipeline      # Ingestão Bronze
python -m src.silver.pipeline      # Tratamento e qualidade (Silver)
python -m src.gold.pipeline        # Datasets analíticos (Gold)
python -m tests.test_silver_gold  # Testes de lógica (sem AWS)
```

Pré-requisitos: arquivo .env com credenciais AWS e S3_BUCKET_NAME, e a camada Bronze já ingerida no S3.

9. Status e validação

- **8/8 testes** de lógica passando (transformações, regras de qualidade e datasets Gold).
- Fase 1 (Bronze), Fase 2 (Silver) e Fase 3 (Gold) concluídas.
- Próximas fases: streaming (Kafka) e análise exploratória / apresentação.